

# Plan d'optimisation du cycle de release — équipe Orion

---

**Livrable 2/2 de l'itération.** Ce document part du cycle décrit par les deux équipes dans leurs sondages, nomme ce qui coûte, et propose un ordre de travaux avec un porteur, un effort et une preuve d'atteinte pour chaque action.

Il se lit avec son jumeau, [schema-architecture.md](#), qui décrit la cible. Ici on décrit **le chemin**.

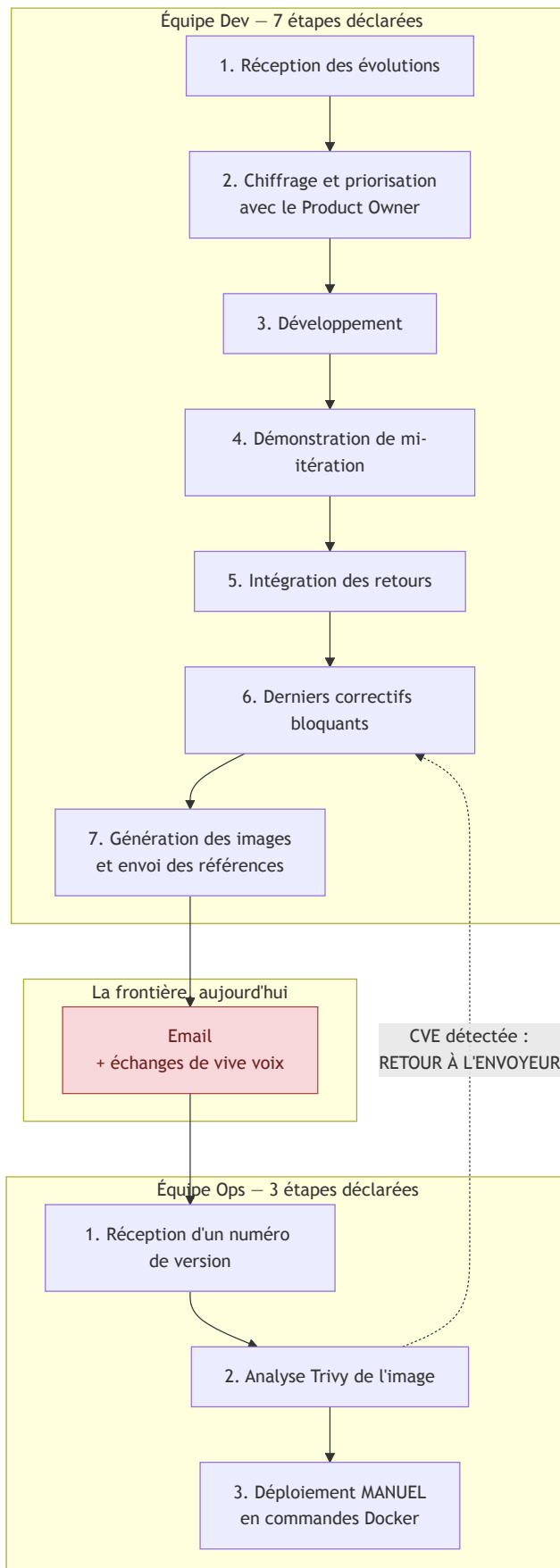
**Principe directeur.** Une optimisation qui ne se mesure pas est une opinion. Chaque action porte donc une preuve observable — un job qui échoue, un chiffre qui bouge, une commande qui rend un résultat — et non un état d'esprit.

---

# 1. Le point de départ, tel que les équipes le décrivent

---

## 1.1 Le cycle actuel



Ce cycle a une qualité qu'il faut reconnaître avant de le critiquer : **il existe et il est décrit**. Une démonstration de mi-itération, un chiffrage avec le Product Owner, un contrôle de sécurité avant déploiement — beaucoup d'équipes au deuxième sprint n'ont pas cela.

Ce qu'il n'a pas, c'est un **point de jonction outillé**. Tout ce qui traverse la frontière traverse une boîte mail.

## 1.2 Le chiffre qui cadre tout le reste

Le collecteur d'indicateurs DORA (`scripts/ci/collect_dora.py`), mesuré le **2026-09-23** sur les **50 pipelines** des trente derniers jours, puis rejoué le 2026-10-02 sur 57 pipelines avec **les mêmes valeurs** — aucun déploiement n'ayant eu lieu entre les deux :

| Indicateur DORA              | Valeur mesurée             | Lecture                                                 |
|------------------------------|----------------------------|---------------------------------------------------------|
| Fréquence de déploiement     | <b>0,1667</b> / jour       | 5 déploiements réussis, concentrés sur deux journées    |
| Délai de mise en production  | <b>1,38 h</b><br>(médiane) | 5 observations ; le déclenchement est manuel            |
| Temps de rétablissement      | <b>2,14 h</b><br>(médiane) | 2 observations                                          |
| Taux d'échec des changements | <b>66,67 %</b>             | 6 échecs sur 9 tentatives, dont 2 comptés comme annulés |

**Ce tableau a changé le 2026-09-22, et c'est le fait marquant de l'itération.** Il affichait jusque-là `0,0` déploiement par jour, deux `null` et 100 % d'échec sur 7 tentatives. La vague 0 de ce plan est donc largement derrière nous : un runner auto-hébergé a levé la contrainte de quota, et trois défauts d'accès au cluster ont été corrigés dans la foulée (détail en A0.1 et dans `MONITORING.md §9.1`).

`null` et `0` ne disent toujours pas la même chose, et le collecteur refuse de les confondre : un délai rendu en `0` afficherait la performance parfaite là où il n'y aurait pas eu de mise en production. La règle ne se voit plus dans cette sortie ; elle redevient visible dès qu'on interroge une fenêtre sans déploiement.

**Conséquence directe sur ce plan :** la question n'est plus « la chaîne aboutit-elle ? » mais « à quel prix ? ». Avec **deux tentatives sur trois qui échouent**, optimiser la vitesse du cycle reste prématuré : on fiabilise d'abord, on accélère ensuite. La vague 1 devient donc l'entrée réelle du plan, et les quatre chiffres ci-dessus deviennent une ligne de base — ce qu'ils ne pouvaient pas être tant qu'ils valaient `null`.

---

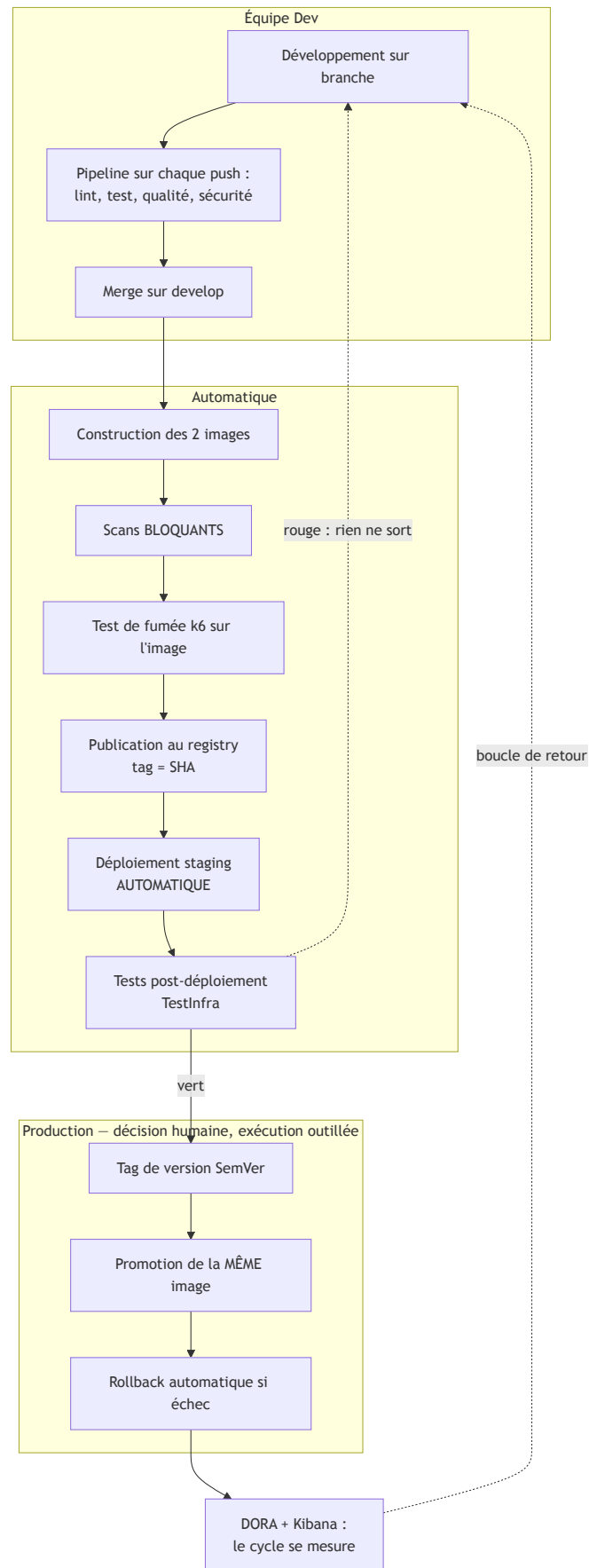
## 2. Les irritants, et ce qu'ils coûtent

Les huit irritants ci-dessous sont soit cités par les équipes, soit établis en lisant le dépôt. La colonne « établi par » distingue les deux, parce qu'un irritant vécu et un irritant constaté ne se traitent pas de la même façon : le premier a déjà l'adhésion de l'équipe, le second doit d'abord être partagé.

| #  | Irritant                                                           | Établi par                     | Ce qu'il coûte aujourd'hui                                                                                                                                                            |
|----|--------------------------------------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I1 | Deux déploiements sur trois échouent depuis la CI                  | Dépôt — DORA                   | <b>Requalifié le 2026-09-22</b> : la chaîne aboutit (5 réussites sur 9), elle n'est pas fiable. Taux d'échec à 66,67 %, sur 9 observations seulement.                                 |
| I2 | Les CVE se découvrent <b>chez Ops</b> , après remise               | Les deux sondages              | Le premier déploiement de démonstration a été retardé. Retravail non planifié en fin d'itération, au pire moment.                                                                     |
| I3 | Les scans existent dans la CI mais <b>ne bloquent rien</b>         | Dépôt                          | <code>trivy-fs</code> et les scans d'image tournent en <code>-exit-code 0</code> . Une image vulnérable est publiée comme une image saine. I2 semble traité alors qu'il ne l'est pas. |
| I4 | Le déploiement est <b>manuel</b> , en commandes Docker             | Sondage Ops                    | Non reproductible, non traçable, indisponible quand la personne qui sait ne l'est pas. Cité comme irritant n°1 par l'équipe Ops.                                                      |
| I5 | <b>Rupture de parité</b> : HyperSQL en dev, PostgreSQL en prod     | Croisement des deux sondages   | Le code n'a jamais vu sa base de production. Le schéma est généré par Hibernate sans migration. Le back est plafonné à 1 replica.                                                     |
| I6 | La frontière Dev↔Ops est un <b>email</b>                           | Sondage Ops                    | Aucune traçabilité, aucun horodatage, aucune réponse rapide à « quelle version tourne ? ».                                                                                            |
| I7 | <b>Trois artefacts</b> dont une image tout-en-un                   | Sondage Dev                    | Couple les cadences front et back, empêche la montée en charge, triple la surface à scanner.                                                                                          |
| I8 | <b>Aucune compétence Kubernetes</b> déclarée dans les deux équipes | Absence dans les deux sondages | La cible repose sur une compétence que personne n'a. Risque non pas de lenteur, mais d'incident non diagnosticable.                                                                   |

**Trois irritants sont traités et il serait malhonnête de les recompter.** I7 est réglé : l'image tout-en-un a été scindée au commit `2931df3` puis supprimée au commit `f125ef0`. I3 est réglé depuis le 2026-09-19 : les scans bloquent (A1.1). I2 l'est donc aussi, avec une réserve que le 2026-10-02 a mise au jour : l'une des portes, Dependency-Check, n'analysait aucun jar et « passait » sur du vide (A1.3). La colonne « ce qu'il coûte aujourd'hui » garde la description d'origine de ces irritants : c'est ce qui permet de mesurer l'écart.

### 3. Le cycle cible



Trois différences de nature avec le cycle du §1.1, et une seule qui compte vraiment :

1. La sécurité **arrête** le pipeline au lieu de le commenter.
2. Le déploiement en staging est **déclenché par le pipeline**, pas par une personne.
3. **Le seul acte humain restant est la décision de mettre en production**. Ce n'est pas une automatisation incomplète, c'est le point d'arrêt voulu : l'automatisation doit supprimer le geste répétitif, pas la décision.

---

## 4. Le plan, par vagues

---

Chaque action porte un identifiant, l'irritant traité, un porteur, un effort ( $S \leq 1$  j,  $M \leq 3$  j,  $L > 3$  j) et **une preuve d'atteinte** — ce qu'on regarde pour dire que c'est fait.

**État des vagues au 2026-10-02.** « Écrit » veut dire présent dans le dépôt mais jamais exécuté par un pipeline : le travail de ce jour-là n'est ni engagé ni passé dans un runner.

| Vague | Actions faites                  | En partie, ou écrites sans avoir tourné | Pas commencées                  |
|-------|---------------------------------|-----------------------------------------|---------------------------------|
| 0     | A0.1, A0.2, A0.3 — <b>close</b> | —                                       | —                               |
| 1     | A1.1, A1.2, A1.3                | —                                       | A1.4 ; A1.5 n'est pas mesurable |
| 2     | A2.1                            | —                                       | A2.2, A2.3, A2.4, A2.5          |
| 3     | —                               | A3.3 (écrite)                           | A3.1, A3.2, A3.4, A3.5          |
| 4     | A4.3                            | A4.2 (Release sans changelog)           | A4.1, A4.4                      |

Hors plan, deux chantiers ont avancé et changent la lecture des vagues 3 et 4 : **huit règles d'alerte Kibana** couvrent désormais disponibilité, performance et sécurité en staging, sans notification hors de Kibana (`MONITORING.md` §11) ; et les **traces de l'API** sont écrites mais pas déployées (§10 du même document).

### Vague 0 — Faire aboutir la chaîne une fois — close le 2026-09-22

C'était le préalable à tout le reste : tant qu'elle n'était pas close, les vagues suivantes optimisaient quelque chose dont on ignorait si cela fonctionnait. Les trois actions sont réalisées ; elles restent inscrites ici avec leur preuve, parce qu'un plan dont on efface les lignes faites ne se relit plus.

| ID   | Action                                                                                                          | Irritant | Porteur      | Effort | Preuve d'atteinte                                                                                                                        |
|------|-----------------------------------------------------------------------------------------------------------------|----------|--------------|--------|------------------------------------------------------------------------------------------------------------------------------------------|
| A0.1 | Rétablir une capacité d'exécution CI : runner auto-hébergé sur le poste Ops, ou remise à niveau du quota GitLab | I1       | Nico         | M      | <b>Fait le 2026-09-22</b> — runner auto-hébergé enregistré sur le poste. Plus un pipeline ne s'arrête sur <code>ci_quota_exceeded</code> |
| A0.2 | Obtenir <b>un</b> déploiement staging réussi depuis la CI, de bout en bout                                      | I1       | Nico + Temim | M      | <b>Fait le 2026-09-22 à 18:47</b> — <code>deploy-staging</code> en vert sur <code>develop</code> , après trois échecs le même jour       |
| A0.3 | Rejouer le collecteur DORA après A0.2                                                                           | I1       | Josefina     | S      | <b>Fait le 2026-09-23</b> — <code>deployment_frequency</code> à 0,1667 et plus aucun indicateur à <code>null</code> (§1.2)               |

**Ce que la vague 0 a coûté, et qu'il faut retenir.** Le runner n'a pas suffi. Il a surtout rendu visibles trois défauts que personne n'avait pu rencontrer, faute de job allant assez loin : un kubeconfig de poste désignant `127.0.0.1`, inatteignable depuis un conteneur ; un RBAC d'agent qui ne couvrait aucun objet applicatif ; une variable `$STAGING_NAMESPACE` absente, qui faisait déployer dans `default` sans rien signaler. Les trois sont corrigés, le dernier par un garde-fou qui fait échouer le job. C'est la démonstration la plus nette du principe directeur de ce document : **une chaîne qu'on n'exécute pas ne révèle pas ses défauts, elle les conserve.**

**Pourquoi A0.2 restait confiée à un binôme Ops + junior.** C'était le premier déploiement réussi du projet : le faire à deux met Temim au contact du geste que personne dans l'équipe n'avait encore accompli, et donne à Nico un témoin de ce qui casse. Les trois échecs du 22 septembre, diagnostiqués l'un après l'autre avant la réussite de 18:47, sont précisément le matériel que ce binôme devait voir passer. La montée en compétence I8 commence ici, pas en vague 3.

**⚠ Close ne veut pas dire acquise.** Cinq déploiements réussis sur neuf tentatives, sur deux journées : la chaîne aboutit, elle n'est pas fiable. Le bloquant suivant n'est plus l'exécution, c'est le taux d'échec — ce que traitent la vague 1 et A3.1.

## Vague 1 — Rendre les contrôles contraignants

| ID   | Action                                                                                                                   | Irritant            | Porteur        | Effort | Preuve d'atteinte                                                                                                                                                                                                                                                                                                                       |
|------|--------------------------------------------------------------------------------------------------------------------------|---------------------|----------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A1.1 | Passer les scans Trivy en mode bloquant sur HIGH, CRITICAL (trivy-fs, package-back, package-front)                       | I3, I2              | Maïa           | S      | <b>Fait le 2026-09-19.</b> Une image porteuse d'une CVE critique fait échouer package-*. Depuis le 2026-10-02 (écrit) : le scan d'image a lieu <b>avant</b> le push, et chaque scan publie un rapport JSON en artefact (scripts/ci/trivy_scan.sh)                                                                                       |
| A1.2 | Tenir un fichier d'exceptions <b>daté et justifié</b> : une entrée = un identifiant, un chemin, une raison, une date     | I3                  | Maïa + Roubina | S      | <b>Fait le 2026-09-19 — .</b> trivyignore.yaml, 7 entrées au 2026-10-02, bornées par chemin, revue au 2026-12-31 ; et depuis le 2026-10-02, suppressions.xml pour Dependency-Check : 12 CVE de Spring Framework, même échéance                                                                                                          |
| A1.3 | Rendre dependency-check-back bloquant au-delà d'un score CVSS convenu                                                    | I2                  | Sylvain        | S      | <b>Bloquant depuis le 2026-09-19, effectif depuis le 2026-10-02 seulement.</b> Le job n'analysait aucun jar (skipTestGroups) ; il en lit 82, et a échoué en local sur 13 CVE réelles avant traitement. La preuve prévue — une dépendance vulnérable introduite exprès — n'a pas été jouée : c'est une vraie qui a fait la démonstration |
| A1.4 | Miroiter les images de base dans le registry interne et n'y référencer qu'elles                                          | I6, souhaite Ops C8 | Nico           | M      | Aucun FROM ne pointe vers DockerHub ; le build passe DockerHub coupé                                                                                                                                                                                                                                                                    |
| A1.5 | Remplacer l'email par le pipeline comme canal : un déploiement se désigne par un tag d'image, jamais par un numéro dicté | I6                  | Roubina + Nico | S      | Aucun échange de version par email sur une itération complète                                                                                                                                                                                                                                                                           |



**A1.3 enseigne ce qu'une preuve d'atteinte doit regarder.** Le job était bloquant et vert ; la case aurait pu être cochée le 2026-09-19. Il ne lisait rien. Une porte bloquante qui ne lit rien ne se distingue pas, à l'œil, d'une porte qui n'a rien trouvé — la preuve d'atteinte d'un contrôle doit donc inclure ce qu'il a regardé, pas seulement son verdict.

**A1.1 tient en un caractère, et c'est précisément le piège.** Le travail n'est pas la modification, c'est A1.2 : sans une liste d'exceptions tenue honnêtement, un scan bloquant devient un scan qu'on contourne, et l'équipe apprend à ignorer un signal rouge. Les deux actions sont indissociables ; les livrer séparément serait pire que ne rien faire.

## Vague 2 — Réparer la parité d'environnement

| ID   | Action                                                                                          | Irritant | Porteur           | Effort | Preuve d'atteinte                                                                                                                  |
|------|-------------------------------------------------------------------------------------------------|----------|-------------------|--------|------------------------------------------------------------------------------------------------------------------------------------|
| A2.1 | Faire tourner les tests d'intégration du back contre un <b>PostgreSQL réel</b> en service de CI | I5       | Sylvain           | M      | <b>Fait le 2026-09-04</b> (commit ffe1d39) — test-back et mutation-back tournent sur un service postgres:16-alpine (QUALITY.md §7) |
| A2.2 | Introduire Flyway ou Liquibase et <b>retirer ddl-auto</b> en environnement persistant           | I5       | Sylvain + Roubina | L      | Une évolution d'entité produit un script de migration, pas une recréation de schéma                                                |
| A2.3 | Déployer PostgreSQL en <b>StatefulSet</b> + PVC, identifiants en <b>Secret</b>                  | I5       | Nico              | M      | La base survit à la suppression du pod applicatif                                                                                  |
| A2.4 | Lever le plafond de 1 replica sur le back et le vérifier sous charge k6                         | I5       | Maïa + Temim      | S      | 2 replicas servent des données cohérentes sous <b>k6-load</b>                                                                      |
| A2.5 | Écrire et <b>jouer</b> la procédure de sauvegarde/restauration                                  | I5       | Nico              | M      | Une restauration effective, pas un <b>CronJob</b> jamais rejoué                                                                    |

**L'ordre est contraint et ne se négocie pas.** A2.2 avant A2.3 : mettre en place une base persistante en gardant **ddl-auto** revient à installer un mécanisme qui détruira des données à la première évolution d'entité. La commodité tolérable sur une base jetable devient un danger dès qu'elle persiste.

## Vague 3 — Automatiser le déploiement et combler l'angle mort

| ID   | Action                                                                                          | Irritant | Porteur                      | Effort | Preuve d'atteinte                                                                                                                                                                                                                                                                     |
|------|-------------------------------------------------------------------------------------------------|----------|------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A3.1 | Passer <code>deploy-staging</code> de <code>when: manual</code> à <code>when: on_success</code> | I4       | Nico                         | S      | Un merge sur <code>develop</code> met à jour staging sans intervention                                                                                                                                                                                                                |
| A3.2 | Tests post-déploiement en <b>TestInfra</b> — l'outil que l'équipe Ops maîtrise déjà             | I4, I8   | Maïa                         | M      | Un déploiement fonctionnel mais cassé fait échouer le job                                                                                                                                                                                                                             |
| A3.3 | Exécuter le collecteur DORA <b>dans la CI</b> au lieu de le lancer à la main                    | I1       | Josefina                     | S      | <b>Écrit le 2026-10-02, jamais exécuté</b> : job <code>dora-metrics</code> sur <code>develop</code> , <code>main</code> et les tags, artefact <code>reports/dora.json</code> . Il n'alimente pas Elasticsearch, injoignable depuis un job : le tableau de bord reste rempli à la main |
| A3.4 | Montée en compétence Kubernetes : deux ateliers, un incident simulé, une astreinte en binôme    | I8       | Nico (anime), toute l'équipe | L      | Chaque membre a diagnostiqué seul un pod en <code>CrashLoopBackOff</code> et déclenché un rollback                                                                                                                                                                                    |
| A3.5 | Documenter les décisions structurantes en ADR courtes                                           | I8       | Josefina                     | S      | Une ADR par décision d'architecture non triviale                                                                                                                                                                                                                                      |

**Sur A3.2 et le choix de TestInfra.** Le réflexe serait d'écrire ces tests dans l'outillage du pipeline. On les écrit en TestInfra parce que l'équipe Ops le note « bon » et qu'elle sera celle qui les lira à 3 h du matin. Un test de production que son lecteur ne sait pas déchiffrer ne sera pas maintenu — et un test non maintenu finit désactivé.

**Sur A3.4 et son critère de sortie.** « Se former à Kubernetes » n'est pas un critère ; « avoir diagnostiqué seul un `CrashLoopBackOff` et déclenché un rollback » en est un. C'est la seule ligne du plan dont l'effort est en semaines, et c'est aussi celle qui protège de l'incident le plus coûteux : celui que personne ne sait lire.

## Vague 4 — Réduire le risque de la mise en production

| ID   | Action                                                   | Irritant | Porteur     | Effort | Preuve d'atteinte                                                                                                                                                                                                                       |
|------|----------------------------------------------------------|----------|-------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A4.1 | Déploiement progressif — canary ou blue/green            | I4       | Nico + Maïa | L      | Une version fautive n'atteint qu'une fraction du trafic                                                                                                                                                                                 |
| A4.2 | Changelog généré depuis les Conventional Commits         | I6       | Josefina    | S      | <b>En partie, écrit le 2026-10-02</b> : le job <code>release</code> crée une Release GitLab sur chaque tag, avec le commit et les images — pas encore la liste des changements. Jamais exécuté : aucun tag n'a abouti                   |
| A4.3 | Notification d'échec de déploiement sur un canal partagé | I6       | Temim       | S      | <b>Fait le 2026-09-29</b> (commit <code>44a1418</code> ) — <code>notify-echec</code> et l' <code>after_script</code> des déploiements, vers <code>NOTIFY_WEBHOOK_URL</code> . Sans la variable, le message reste dans le journal du job |
| A4.4 | Signature des images et attestation de provenance        | I2       | Maïa        | M      | Une image non signée est refusée au déploiement                                                                                                                                                                                         |

## 5. Comment on saura que le plan a produit un effet

Quatre indicateurs DORA, plus quatre propres aux irritants d'Orion. La colonne « aujourd'hui » n'est pas une estimation : c'est ce qui est mesuré, ou l'indication franche qu'il n'y a pas de mesure.

| Indicateur                  | Aujourd'hui       | Après vague 1               | Après vague 3       | Comment on le mesure         |
|-----------------------------|-------------------|-----------------------------|---------------------|------------------------------|
| Fréquence de déploiement    | <b>0,1667 / j</b> | > 0,33 / j                  | ≥ 1 / j sur staging | <code>collect_dora.py</code> |
| Délai de mise en production | <b>1,38 h</b>     | < 1,38 h                    | < 1 j, sans clic    | <code>collect_dora.py</code> |
| Temps de rétablissement     | <b>2,14 h</b>     | mesurable sur plus de 2 cas | < 1 h               | <code>collect_dora.py</code> |

| Indicateur                                     | Aujourd'hui                                         | Après vague 1  | Après vague 3 | Comment on le mesure                                       |
|------------------------------------------------|-----------------------------------------------------|----------------|---------------|------------------------------------------------------------|
| Taux d'échec des changements                   | <b>66,67 %</b>                                      | < 50 %         | < 15 %        | <code>collect_dora.py</code>                               |
| Délai entre remise et retour CVE               | <b>minutes</b> (le job échoue) depuis le 2026-09-19 | <b>minutes</b> | idem          | Durée du job <code>package-*</code>                        |
| Part d'opérations de déploiement manuelles     | <b>100 %</b>                                        | 100 %          | staging à 0 % | Comptage des gestes <code>when: manual</code>              |
| Artefacts publiés par release                  | <b>2</b> (3 avant <code>f125ef0</code> )            | 2              | 2             | <code>docker images</code> du registry                     |
| Environnements où le code voit sa base de prod | <b>1</b> (CI, depuis A2.1)                          | 1              | 1             | Présence du service PostgreSQL dans <code>test-back</code> |

Trois honnêtetés à maintenir dans ce tableau. **Un indicateur sans donnée se rend `null`, jamais `0`** — c'est la règle du collecteur, et l'afficher en `0` transformerait une absence de mesure en performance parfaite. **La part d'opérations manuelles reste à 100 % après la vague 1** : rendre un scan bloquant ne déploie rien, et le déclenchement des deux jobs de déploiement reste un clic. Une case qui ne bouge pas au bon moment est le signe que le tableau mesure quelque chose de réel.

Et la troisième, qui porte sur la colonne « aujourd'hui » elle-même : **ces valeurs reposent sur neuf tentatives étalées sur deux jours**. Elles constituent une ligne de base parce qu'il en faut une, pas parce qu'elles seraient stables. La cible « après vague 1 » d'un délai inférieur à 1,38 h ne vaudra d'ailleurs que si elle se compare à un nombre d'observations comparable. S'y ajoute une limite du collecteur, assumée et documentée en `MONITORING.md` §9.3 : un rollback qui a échoué compte quand même comme une annulation, donc le taux d'échec est plutôt au-dessus de la réalité qu'en dessous.

## 6. Ce que le plan ne traite pas, et pourquoi

- **L'authentification de l'application.** MicroCRM n'en a aucune. C'est un sujet produit, pas un sujet de cycle de release ; il doit être ouvert ailleurs, et avant tout usage réel.
- **Le passage au cloud.** L'architecture reste locale. Le chiffage AWS/Azure n'existe pas, et un plan qui promettrait une bascule sans coût connu ne serait pas un plan.
- **La sécurisation de la stack ELK.** `xpack.security` est désactivé et Kibana n'a pas d'Ingress ; assumé pour une stack de démonstration locale, inacceptable ailleurs. À rouvrir si elle sort du poste.

- **La vérification des sources Mermaid dupliquées.** Rien ne compare les blocs des documents et les fichiers `docs/schemas/*.mmd`. Le coût d'un test dépasse aujourd'hui celui d'une relecture — mais c'est un choix, pas un oubli.

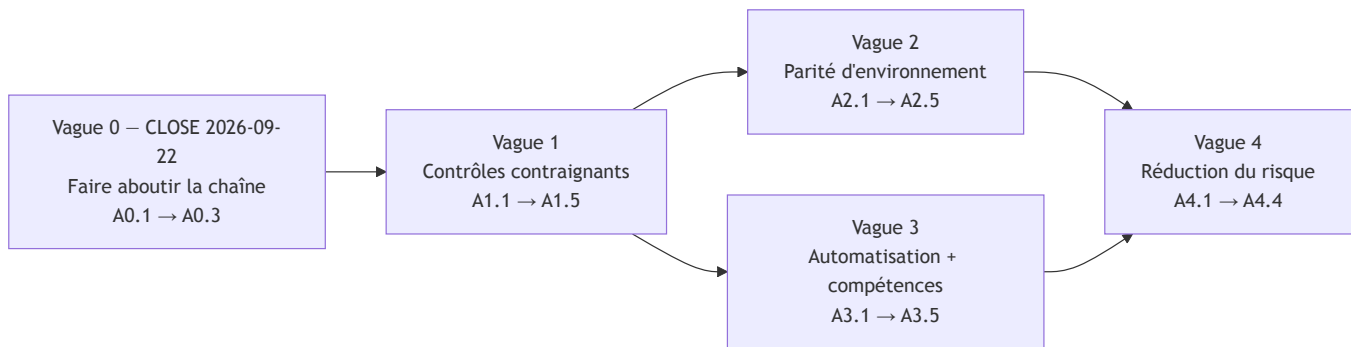
## 7. Les risques du plan lui-même

Un plan qui n'énonce pas ses propres risques demande qu'on lui fasse confiance.

| Risque                                                                                               | Probabilité      | Impact          | Ce qu'on fait                                                                                                 |
|------------------------------------------------------------------------------------------------------|------------------|-----------------|---------------------------------------------------------------------------------------------------------------|
| <b>A1.1 sans A1.2</b> : le blocage devient un obstacle qu'on contourne                               | élevée           | fort            | Les deux actions sont livrées ensemble, et la liste d'exceptions est relue à chaque itération                 |
| <b>La charge repose sur deux personnes</b> côté Ops                                                  | élevée           | fort            | A3.2 et A3.4 transfèrent volontairement du travail vers l'équipe Dev ; A0.2 est en binôme dès le premier jour |
| <b>La vague 2 déborde.</b> Flyway sur une base existante est un chantier, pas une tâche              | moyenne          | moyen           | A2.1 seule apporte déjà l'essentiel du bénéfice et se livre indépendamment                                    |
| <b>La compétence Kubernetes ne monte pas</b> faute de temps dédié                                    | moyenne          | <b>critique</b> | A3.4 porte un critère de sortie observable ; sans lui, le sujet se reporte indéfiniment                       |
| <b>Le runner auto-hébergé tombe ou change de poste</b> — la contrainte a été déplacée, pas supprimée | moyenne          | fort            | A0.1 l'a mis sous le contrôle de l'équipe ; reste à ne pas dépendre d'une seule machine, et de qui l'allume   |
| <b>Josefina termine son stage</b> avant la fin des actions qui lui sont confiées                     | certaine à terme | faible          | Ses actions sont toutes en effort S et documentées par des ADR (A3.5)                                         |

## 8. Ordre d'exécution résumé

---



Les vagues 2 et 3 sont **parallélisables** : la première est un chantier back porté par Sylvain, la seconde un chantier plateforme porté par Nico. Elles se rejoignent avant la vague 4, parce qu'un déploiement progressif sur une base non persistante ne prouverait rien.

La vague 0, elle, ne se parallélisait pas — c'était le seul point du plan sur lequel il n'y avait pas d'arbitrage possible, et elle est close depuis le 2026-09-22. Ce qu'elle a démontré tient en une phrase, qui vaut pour la suite : **une chaîne qu'on n'exécute pas ne révèle pas ses défauts, elle les conserve**. Trois défauts attendaient derrière le quota ; rien ne dit que la vague 1 n'en découvrira pas d'autres en rendant les contrôles bloquants.